MPMC — Module 2 · PART 1: 8086 ARCHITECTURE — EXAM-READY NOTES

Topic: Generations · Functional blocks · Pin diagram & signals · BIU–EU architecture · Pipelining · Registers · Flags · Memory segmentation & physical address · Interrupts

Source: Module 2 lecture slides (ECE3004 – Microprocessors and Microcontrollers).

Module 2 is split into 3 files — use them together, nothing is left out:

- **Part 1 (this file)** → Architecture: generations, pins, BIU/EU, registers, flags, physical address, interrupts
- **Part 2** → `MPMC_Module2_Part2_Addressing_Modes.md` — all 12 addressing modes
- **Part 3** → `MPMC_Module2_Part3_Instruction_Set.md` — instruction set, assembler directives, procedures/macros, programs
- (All-in-one master copy: `MPMC_Module2_Exam_Notes.md`)

CONTENTS OF THIS FILE

1. Generations of microprocessors
2. General microprocessor functional blocks
3. 8086 microprocessor — overview & salient features
4. 8086 pin diagram and signals (common / minimum / maximum mode)
5. Inside the 8086 — BIU + EU architecture and pipelining
6. Registers of 8086 and the flag register
7. Memory segmentation & physical address generation
8. 8086 interrupts — sources, processing, IVT, groups, types 0–4
9. Most likely exam questions (architecture)
10. Last-minute revision cram sheet (architecture)

1. GENERATIONS OF MICROPROCESSORS

Generation	Period	Technology	Word size / Pins	Key features	Example
First	1971 – 1973	**PMOS** (not TTL compatible)	4-bit → 16 pins; 8- & 16-bit → 40 pins	Signals had to be **multiplexed** due to pin limitations	Intel 4004/4040
Second	1973 onward	**NMOS** (faster, higher density, TTL compatible)	4 / 8 / 16-bit → 40 pins	Ability to address large memory spaces and I/O ports, greater levels of subroutine nesting, better interrupt handling	**Intel 8085 (8-bit)**
Third	1978 onward	**HMOS** (faster, higher packing density)	16-bit → 40 / 48 / 64 pins	Easier to program, dynamically relocatable programs, **multiply/divide arithmetic hardware**, more powerful interrupt handling, flexible I/O port addressing	**Intel 8086 (16-bit)**
Fourth	1980s	**HCMOS** (low-power version of HMOS)	32-bit	Physical memory space 2^{24} = **16 MB**, virtual memory space 2^{40} = **1 TB**, **floating-point hardware**, increased number of addressing modes	Intel 80386
Fifth	—	—	32/64-bit	Advanced pipelining, superscalar	**Pentium**

Memory trick: P-N-H-H-P = PMOS → NMOS → HMOS → HCMOS → Pentium.

2. GENERAL MICROPROCESSOR FUNCTIONAL BLOCKS

Block	Function
ALU (Computational Unit)	Performs arithmetic and logic operations
Flag Register	Stores various conditions of the results as status bits called **flags**
Register array / internal memory	Internal storage of data
PC / IP	Generates the address of the **instructions to be fetched** from memory and sends it through the **address bus**
Instruction decoding unit	Decodes instructions; sends information to the timing and control unit
Timing and control unit	Generates control signals for internal and external operations of the microprocessor

Buses: Control bus, **Address bus**, **Data bus** interconnect these blocks with memory and I/O.

3. 8086 MICROPROCESSOR — OVERVIEW & SALIENT FEATURES

- **First 16-bit processor** released by **INTEL in the year 1978**.
- Originally **HMOS**, now manufactured using the **HMOS III** technique.
- Approximately **29,000 transistors**, **40-pin DIP**, **+5 V** supply.
- **Does not have an internal clock** → needs an **external asymmetric clock source with 33 % duty cycle** (supplied by the **8284A** clock generator).
- **20-bit address to access memory** → can address up to **2^{20} = 1 megabyte** of memory space.
- 16-bit data bus; word size = 16 bits.
- Two operating modes: **Minimum mode** (no coprocessors, single processor) and **Maximum mode** (multiprocessor/coprocessor configurations).

Quick comparison table (frequently asked):

Feature	8085	8086
Word length	8-bit	16-bit
Address bus	16-bit	20-bit
Memory	64 KB	1 MB
Clock	3 MHz	5 / 8 / 10 MHz (version dependent)
Multiplexing	Lower address/data	Address/data + status multiplexed
Pipelining	No	**Yes (6-byte instruction queue)**

4. 8086 PIN DIAGRAM AND SIGNALS (40-pin DIP)

Pin layout (learn the groups):

Pins	Name
1, 20	GND (ground)
2–16, 39	**AD0–AD15** (address/data bus, bidirectional, multiplexed)
35–38	**A16/S3, A17/S4, A18/S5, A19/S6**
17	**NMI**
18	**INTR**

Pins	Name
19	**CLK**
21	**RESET**
22	**READY**
23	**TEST**
24	**QS1**
25	**QS0** (max mode) / **ALE** (min mode)
26	**S0** (max) / **DEN** (min)
27	**S1** (max) / **DT/R** (min)
28	**S2** (max) / **M/IO** (min)
29	**LOCK** (max) / **WR** (min)
30	**RQ/GT1** (max) / **HLDA** (min)
31	**RQ/GT0** (max) / **HOLD** (min)
32	**RD**
33	**MN/MX** (minimum/maximum mode select)
34	**BHE/S7**
40	VCC

Exam tip: The same pin carries **different names in minimum and maximum modes** — this is the most repeated pin question.

4.1 Common Signals (both modes)

Signal	Direction	Description
AD0–AD15	Bidirectional, multiplexed	Low-order address bus multiplexed with data. Written **A0–A15** when carrying memory addresses, **D0–D15** when carrying data.
A16/S3 – A19/S6	Output, multiplexed	High-order address bus multiplexed with status signals.
BHE/S7 (active low)	Output	**Bus High Enable** — enables data onto the **most significant half of the data bus (D8–D15)**. 8-bit devices connected to the upper half of the data bus use BHE. Multiplexed with status signal S7.
MN/MX	Input	Indicates the mode the processor is to operate in: **High (1) → Minimum mode; Low (0) → Maximum mode**.
RD (active low)	Output	Read operation; active low.
READY	Input, active **HIGH**	Acknowledgement from a **slow device or memory** that it has completed the data transfer. The signal made available by the devices is **synchronised by the 8284A clock generator** to provide the ready input to the 8086.
RESET	Input, active **HIGH**	Causes the processor to immediately terminate its present activity. Must be **active HIGH for at least four clock cycles**.
CLK	Input	Provides the basic timing for processor operation and bus control activity. An **asymmetric square wave with 33 % duty cycle**.
INTR	Input, active HIGH, triggered	**Interrupt Request** — sampled during the **last clock cycles of each instruction** to determine the availability of the request. If any interrupt request is pending, the processor enters the **interrupt acknowledge cycle**. Internally synchronised.
NMI	Input	Non-maskable interrupt (positive-edge signal).
TEST	Input	Used with the WAIT instruction; the processor waits while TEST is inactive.

Note: The 8086 does **not** have an on-chip clock generation circuit → the **8284 clock generator chip** is connected to the CLK pin of the 8086.

4.2 Minimum-Mode Signals (MN/MX = 1)

Signal	Description
ALE (Address Latch Enable)	Used to **demultiplex the address and data lines** using external latches (8282). A positive pulse is generated each time the processor begins any operation and indicates the availability of a **valid address** on the address/data lines. Also used to differentiate memory access and I/O access.
DEN (Data Enable)	Output signal from the processor used as **output enable for the transceivers** (**8286**). The transceiver is a device used to **separate data from the address/data bus**.
DT/R (Data Transmit/Receive)	Output signal from the processor to control the **direction of data flow** through the data transceivers.
M/IO	Used to differentiate memory access and I/O access: for **memory reference instructions it is high**; for **IN and OUT instructions it is low**.
WR (active low)	Write control signal; asserted low whenever the processor **writes data to memory or I/O port**.
INTA (active low)	**Interrupt Acknowledge** — when the interrupt request is accepted by the processor, the output is low on this line.
HOLD	Input signal to the processor from the bus masters as a **request to grant control of the bus**; usually used by the **DMA controller** to get control of the bus.
HLDA (Hold Acknowledge)	Acknowledge signal by the processor to the bus master requesting control of the bus through HOLD. **Asserted high** when the processor accepts HOLD.

Demultiplexing hardware (very common exam figure): the **8282 latch** (strobed by ALE) separates **A0–A19** (and produces BHE), while the **8286 transceiver** (controlled by DT/R and DEN) handles **D0–D15**. This converts the multiplexed AD bus into separate address and data buses.

4.3 Maximum-Mode Signals (MN/MX = 0)

Signal	Description
S0, S1, S2 (active low)	Status signals used by the **8288 bus controller** to generate bus timing and control signals.
QS0, QS1	**Queue Status** — the processor provides the status of the queue on these lines; external devices can track the internal status of the queue.
LOCK (active low)	Locks the bus during the next instruction so other bus masters cannot take the bus.
RQ/GT0, RQ/GT1 (Request/Grant)	Bidirectional pins used to request and grant bus access by other processors (e.g. the 8087 coprocessor).

S2 S1 S0 → Machine Cycle (MUST MEMORISE):

S2	S1	S0	Machine Cycle
0	0	0	**Interrupt acknowledge**
0	0	1	Read I/O port
0	1	0	Write I/O port
0	1	1	**Halt**
1	0	0	**Code access** (instruction fetch)
1	0	1	Read memory
1	1	0	Write memory
1	1	1	Passive / Inactive

QS1 QS0 → Queue operation:

QS1	QS0	Queue operation
0	0	No operation
0	1	First byte of an opcode from queue
1	0	**Empty the queue**
1	1	Subsequent byte from queue

4.4 Minimum vs Maximum Mode

Point	Minimum Mode	Maximum Mode
MN/MX pin	High (1)	Low (0)
Coprocessor use	Does not associate with any coprocessor; cannot be used for multiprocessor systems	Works in **multiprocessor / coprocessor** configuration
Bus controller	Not needed — 8086 gives all signals itself	**8288 bus controller** required
Signals generated	ALE, DEN, DT/R, M/IO, WR, INTA, HOLD, HLDA	S0–S2, QS0–QS1, LOCK, RQ/GT0–RQ/GT1
Typical use	Single-processor small systems	Multiprocessor systems (8086 + 8087 + 8089 etc.)

5. INSIDE THE 8086 — BIU + EU ARCHITECTURE AND PIPELINING

5.1 Pipelining

- **Two ways to make a CPU process information faster:**
  1. **Increase the working frequency** — technology dependent.
  2. **Change the internal architecture of the CPU** ← what Intel did.
- **Pipelining = allowing the CPU to fetch and execute at the same time.**
- Intel implemented pipelining by **splitting the internal structure of the 8088/8086 into two sections that work simultaneously**:
  - **Execution Unit (EU)** — executes instructions **previously fetched**.
  - **Bus Interface Unit (BIU)** — **accesses memory and peripherals**.
- **BIU and EU functions are separate** → instruction fetch overlaps execution → speed-up.

5.2 Bus Interface Unit (BIU)

Functions: fetches instructions, reads data from memory and I/O ports, writes data to memory and I/O ports.
Contains:

1. **Dedicated adder** to generate the **20-bit address**.
2. **Four 16-bit segment registers** — **CS, DS, SS, ES**.
3. **Instruction Pointer (IP)**.
4. **6-byte instruction queue** (FIFO).

Segment registers:

Register	Points to	Offset used	Notes
CS (Code Segment)	Base/start of the current **code** segment	**IP**	16-bit. BIU computes the 20-bit physical address by **shifting CS 4 bits left and adding IP**. All instructions of a program are relative to CS × 16 plus the offset provided by IP.
DS (Data Segment)	Current **data** segment	**SI / DI / 16-bit displacement**	16-bit. **Operands for most instructions** are fetched from this segment.
SS (Stack Segment)	Current **stack**	**SP** (PUSH/POP), **BP** (based addressing)	16-bit. Physical stack address = SS × 16_{10} + SP; in based addressing mode the address comes from SS and BP.
ES (Extra Segment)	**Extra** segment — holds data **in excess of 64 K** pointed to by DS	**DI** (string destination)	16-bit. **String instructions use ES:DI** to determine the destination address.

Memory segmentation facts:

- The 8086's **1 MB memory is divided into segments of up to 64 KB each**.
- Programs obtain access to code and data in the segments by **changing the segment register content** to point to the desired segments.
- The 8086 can **directly address four segments (256 KB within the 1 MB)** at any particular time.

Instruction Pointer (IP):

- **16-bit**; always points to the **next instruction to be executed** within the currently executing code segment.
- Contains the **16-bit offset address** pointing to the next instruction code within the 64 KB code segment area.
- Its content is **automatically incremented** as the execution of the next instruction takes place.

Instruction Queue:

- A group of **First-In-First-Out (FIFO)** registers in which **up to 6 bytes of instruction code are pre-fetched** from memory ahead of time.
- Done to **speed up execution by overlapping instruction fetch with execution**.
- This mechanism is known as **pipelining**.

5.3 Execution Unit (EU)

Functions: decodes and executes instructions. A **decoder** in the EU control system translates instructions.
Contains:

- **16-bit ALU** for performing arithmetic and logic operations.

- **Four general-purpose registers (AX, BX, CX, DX)** — each 16-bit.
- **Pointer registers — SP, BP** (16-bit).
- **Index registers — SI, DI** (16-bit).
- **Flag register**.

8-bit splitting: some 16-bit registers can be used as two 8-bit registers:

16-bit	→ 8-bit halves
AX	**AH, AL**
BX	**BH, BL**
CX	**CH, CL**
DX	**DH, DL**

Exam one-liner: BIU fetches, EU executes — and both work simultaneously → pipelining.

6. REGISTERS OF 8086 AND THE FLAG REGISTER

6.1 Register groups

Sl.	Type	Width	Name of register
1	General purpose	16-bit	**AX, BX, CX, DX**
		8-bit	**AL, AH, BL, BH, CL, CH, DL, DH**
2	Pointer	16-bit	**SP, BP**
3	Index	16-bit	**SI, DI**
4	Instruction Pointer	16-bit	**IP**
5	Segment	16-bit	**CS, DS, SS, ES**
6	Flag (PSW)	16-bit	**Flag register**

6.2 Registers and their special functions

Register	Name of the register	Special function
AX	16-bit Accumulator	Stores the 16-bit results of arithmetic and logic operations
AL	8-bit Accumulator	Stores the 8-bit results of arithmetic and logic operations
BX	Base register	Used to hold the **base value in base addressing mode** to access memory data. **Only general-purpose register whose contents can be used for addressing the 8086 memory**; all memory references using BX default to **DS**.
CX	Count register	Used to hold the **count value** in **SHIFT, ROTATE and LOOP** instructions
DX	Data register	Used to hold data for **multiplication and division** operations; also holds the 16-bit port address in indirect I/O
SP	Stack Pointer	Holds the **offset address of the top of stack** memory
BP	Base Pointer	Holds the base value in **base addressing using the SS register** to access data from stack memory
SI	Source Index	Holds the **index value of the source operand (data)** for string instructions
DI	Destination Index	Holds the **index value of the destination operand (data)** for string operations

Extra points from the slides:

- **AX/AL** — the **I/O instructions** use AX or AL for inputting/outputting 16- or 8-bit data to/from an I/O port; **multiplication and division** instructions also use AX or AL.
- **BX** is the only general-purpose register usable for addressing memory (default segment **DS**).
- **CX example:** the instruction L00P START automatically **decrements CX by 1 without affecting flags** and checks if CX = 0. If it is zero, the 8086 executes the next instruction; otherwise the 8086 branches to the label START.
- **SP and BP** are used to access data in the stack segment. **SP** is used as an offset from the current SS during execution of instructions involving the stack; its contents are **automatically updated (incremented/decremented)** by PUSH or POP. **BP** contains an offset address in the current SS, used by instructions utilising the **based addressing mode**.
- **SI and DI** are used in **indexed addressing**. Instructions that process data strings use SI and DI **together with DS and ES respectively** to distinguish between the source and destination addresses.

6.3 Flag Register (PSW) — bit by bit

Bit	Flag	Name	Set (=1) when…
0	**CF**	Carry Flag	There is a **carry out of MSB** in case of addition, or a **borrow** in case of subtraction
2	**PF**	Parity Flag	The **lower byte** of the result contains an **even number of 1's** (odd number of 1's → set to zero)
4	**AF**	Auxiliary Carry Flag	There is a **carry from the lowest nibble (bit 3)** during addition, or a **borrow from bit 3** during subtraction
6	**ZF**	Zero Flag	The result of the computation or comparison performed by an instruction is **zero**
7	**SF**	Sign Flag	The result of any computation is **negative**
8	**TF**	Trap Flag	The processor enters the **single-step execution mode** by generating internal interrupts after the execution of each instruction
9	**IF**	Interrupt Flag	Causes the 8086 to **recognise external maskable interrupts**; clearing IF **disables** these interrupts
10	**DF**	Direction Flag	Used by **string manipulation instructions**. If DF = 0, the string is processed from the **lowest address to the highest (auto-incrementing)**; if DF = 1, from the **highest address towards the lowest (auto-decrementing)**
11	**OF**	Overflow Flag	An **overflow occurs** — i.e. the result of a **signed operation** is too large to be accommodated in the destination register (more than 7 bits for an 8-bit signed operation, more than 15 bits for a 16-bit signed operation)

Bit layout (memorise this):

```
15 14 13 12 11 10  9  8  7  6  5  4  3  2  1  0
            OF  DF  IF TF SF ZF    AF    PF    CF
```

Bits 1, 3, 5, 12, 13, 14, 15 are **undefined/unused** in the 8086.

7. MEMORY SEGMENTATION & PHYSICAL ADDRESS GENERATION

Why needed? The 8086 has **20 address lines** (→ 2^{20} = 1 M byte of memory) but the **largest register is only 16 bits**.

Definitions:

- **Physical Address:** the **actual address of a byte in memory**, i.e. the value which goes out onto the **address bus** (20 bits).
- Memory addresses are represented in the form **Segment : Offset** — e.g. **89AB:F012**.
- **16 bytes of contiguous memory = one "paragraph"** (hence segment × 16_{10}).

Formula:

Physical Address = (Segment register × 16_{10}) + Offset

i.e. shift the segment register content **one hexadecimal place to the left** (same as appending 0) and **add the required offset** to form the 20-bit address.

Worked example — 89AB:F012 → 98AC2H

```
  Segment  89AB  →  89AB0    (paragraph to byte: 89AB × 10H = 89AB0)
  Offset   F012  →   F012    (offset is already in byte units)
                   -----
  Absolute (physical) address   98AC2 H
```

Typical segment : offset pairs:

Access type	Segment	Offset
Instruction fetch (code)	**CS**	**IP**
Stack instructions (PUSH/POP)	**SS**	**SP**
Based addressing (stack data)	**SS**	**BP**
Data (default)	**DS**	BX / SI / DI / displacement
String source	**DS**	**SI**
String destination	**ES**	**DI**

Segment register memory map (slide diagram): four segment registers hold the **upper 16 bits of the starting addresses** of the four memory segments the 8086 is working with at any particular time. Example from the slides: with DS = 2000H, CS = 3000H, SS = 5000H and ES = 7000H, memory shows the Data Segment at 20000H–2FFFFH, Code Segment at 30000H–3FFFFH, Stack Segment at 50000H–5FFFFH and Extra Segment at 70000H–7FFFFH — each **64 KB** long, inside the 1 MB space (up to FFFFFH).

Notation used in the slides (reproduce it in the exam):

- **EA** = Effective Address (the offset part)
- **BA** = Base Address = (segment register) × 16_{10}
- **MA** = Memory Address = BA + EA

The detailed use of these three quantities — with full worked examples — is in **Part 2 (Addressing Modes)**.

8. 8086 INTERRUPTS

8.1 Sources of Interrupts (3 types)

1. An **external signal** applied to the **NMI or INTR** input pin → **(Hardware interrupt)**
2. **Execution of an INTn (n = 00H–FFH) instruction → (Software interrupt)**
3. Interrupt caused by **some error condition** produced in the 8086 instruction execution process — **divide by zero, overflow errors, etc.**

Interrupt classification chart (from slides):

```
8086 Interrupts
├── Hardware Interrupts
│     ├── Maskable Interrupt (INTR)
│     │      → The programmer CAN choose to mask specific interrupts
│     │        and re-enable them later
│     └── Non-maskable Interrupt (NMI)
│            → The programmer CANNOT control when a non-maskable
│              interrupt is served
│            → The processor has to stop the main program to execute
│              the NMI Service Routine
└── Software Interrupts (INT n)
     → 256 types of software interrupts, INT 00 to INT FF
```

- **INTR (pin 18)** — where maskable hardware interrupts are applied.
- **NMI (pin 17)** — non-maskable interrupt.

Uses of Non-Maskable Interrupts:

- Used during **power failure**
- Used during **critical response times**
- Used during **non-recoverable hardware errors**
- Used as a **watchdog interrupt**
- Used during **memory parity errors**

8.2 Interrupt Processing — Steps performed by the 8086

If an interrupt has been requested, the 8086 processes it by performing the following series of steps:

1. **Pushes the content of the flag register onto the stack** to preserve the status of IF and TF flags, by **decrementing the stack pointer (SP) by 2**.
2. **Disables the INTR interrupt by clearing IF** in the flag register.
3. **Resets TF** in the flag register, to disable the single-step (trap) interrupt.
4. **Pushes the content of the Code Segment (CS) register** onto the stack by decrementing SP by 2.
5. **Pushes the content of the Instruction Pointer (IP)** onto the stack by decrementing SP by 2.
6. **Performs an indirect far jump to the start of the Interrupt Service Routine (ISR)** corresponding to the received interrupt.

Flow of steps (slide version):

```
Main Program → Executes the Interrupt instruction → Jumps to the Interrupt Vector Table
→ Takes the CS and IP in the Vector Table → Pushes the existing CS and IP on the Stack
→ Loads the new CS and IP → Jumps to the ISR → Executes ISR
→ Comes back and continues the Main Program
```

Hardware action sequence:

```
Interrupt → Push flag register → Clear IF and TF → Push CS and IP → Load CS and IP
   → [Interrupt Service Routine (ISR) … ends with IRET]
   → Pop IP and CS → Pop flags register → main program continues
```

Remember the order: push **FLAGS → CS → IP**; IRET pops **IP → CS → FLAGS**.

8.3 Interrupt Vector Table (IVT)

- Located at the **bottom of memory**: addresses **00000H – 003FFH** (1 KB).
- Holds **256 interrupt vectors**, each vector = **4 bytes** (2 bytes for **IP** + 2 bytes for **CS**).
- **For an interrupt of type n, the ISR address is stored at:**

Offset = Type number × 4

Example: INT 02H → Offset = 02 × 4 = 08 = 00008H

Layout of one vector (Type 0 shown in the slides):

Memory address	Content
00000H	IP low byte
00001H	IP high byte
00002H	CS low byte
00003H	CS high byte

IVT layout (memory addresses in hex):

Address	Vector
00000H–00003H	Type 00H — Divide by Zero
00004H–00007H	Type 01H — Trap / Single step
00008H–0000BH	Type 02H — NMI
0000CH–0000FH	Type 03H — Break point
00010H–00013H	Type 04H — Overflow
00014H–0007FH	Type 05H ... — Reserved
00080H–00083H	Type 20H — Available
00084H–00087H	Type 21H — Available
...	...
003FCH–003FFH	Type FFH — Available

Order inside each 4-byte vector: lower 2 bytes = IP, upper 2 bytes = CS.

8.4 The 256 Interrupts are Divided into 3 Groups

Group	Types	Purpose
1	**Type 00 to Type 04**	Used for **fixed operations** → called **dedicated interrupts** (5 interrupts)
2	**Type 05 to Type 31**	**Not used by the 8086; reserved for higher processors** like 80286, 80386 etc. (27 interrupts)
3	**Type 32 to Type 255**	**Available for the user** → called **user-defined interrupts**; can be either **H/W interrupts activated through the INTR line** or **S/W interrupts** (224 interrupts)

8.5 Dedicated Interrupts — Types 0 to 4 (MUST MEMORISE)

Type	Name	Description
Type 0	**Divide by Zero Error Interrupt**	The **quotient is large and cannot fit in AL/AX**, or **divide by zero**. (Slide example: `MOV AL,05H / MOV DL,00H / DIV DL / HLT` → "Attempted to divide by zero" error.)
Type 1	**Single step or Trap Interrupt**	Used for **executing the program in single-step mode** by **setting the trap flag**.
Type 2	**Non-Maskable Interrupt (NMI)**	Used for **executing the ISR of the NMI pin** (**positive edge signal**). **NMI cannot be masked by software.**
Type 3	**One-byte INT instruction interrupt**	Used for **providing break points** in the program.
Type 4	**Overflow Interrupt**	Used to handle **any overflow error after signed arithmetic**.

9. MOST LIKELY EXAM QUESTIONS (ARCHITECTURE)

Question	Answer skeleton (write these tables/points)
Draw and explain the architecture of 8086.	Split the diagram into **BIU** (CS/DS/SS/ES segment registers, IP, dedicated 20-bit adder, 6-byte FIFO queue) and **EU** (16-bit ALU, AX/BX/CX/DX, SP/BP, SI/DI, flag register). Explain **pipelining**: BIU fetches the next instruction while the EU executes the current one.
Explain the pin diagram of 8086 / distinguish minimum-mode and maximum-mode pins.	AD0–AD15, A16/S3–A19/S6, BHE/S7, MN/MX, RD, READY, RESET, CLK, INTR, NMI, TEST as common. Min mode: ALE, DEN, DT/R, M/IO, WR, INTA, HOLD, HLDA. Max mode: S0–S2, QS0–QS1, LOCK, RQ/GT0–RQ/GT1. Add the S2S1S0 machine-cycle table and the QS1QS0 queue table.
What are the functions of ALE, DEN and DT/R?	ALE → latch the address (demultiplexing via 8282; valid address indication). DEN → output enable for the 8286 transceiver. DT/R → direction of data flow through the transceiver.
Difference between minimum and maximum mode of 8086.	Use the §4.4 comparison table: MN/MX value, coprocessor use, 8288 requirement, signals generated.
Explain the register organisation of 8086.	4 groups + special functions table of §6 (AX accumulator, BX base, CX count, DX data, SP/BP pointers, SI/DI index, IP, CS/DS/SS/ES, flag register).
Explain the flag register of 8086.	Bit positions: CF (bit 0), PF (2), AF (4), ZF (6), SF (7), TF (8), IF (9), DF (10), OF (11) with the condition that sets each.
How does the 8086 compute the physical address?	PA = (Segment × 16_{10}) + Offset; worked example 89AB:F012 = 98AC2H; explain the 20-bit vs 16-bit register problem and the paragraph concept.
Explain memory segmentation in 8086.	1 MB split into segments of up to 64 KB; 4 active segments (256 KB) at a time; CS/DS/SS/ES with their offsets; memory map example.
What is pipelining? How is it implemented in 8086?	Fetch and execute simultaneously; BIU + EU split; 6-byte instruction queue; benefit = overlapping fetch with execution.
What is the instruction queue?	6-byte FIFO in the BIU that pre-fetches instruction bytes ahead of time to overlap fetch with execution.
List the sources/types of interrupts in 8086.	Hardware (INTR maskable, NMI non-maskable), software (INT n), error conditions (divide by zero, overflow); classification chart.
Explain the interrupt processing steps of the 8086.	Push flags → clear IF → reset TF → push CS → push IP → indirect far jump to ISR (§8.2).

Question	Answer skeleton (write these tables/points)
What is the IVT? How is the interrupt vector address calculated?	00000H–003FFH, 1 KB, 256 vectors × 4 bytes; Offset = Type × 4; lower 2 bytes = IP, upper 2 bytes = CS; example INT 02H → 00008H.
Explain the dedicated interrupts (Types 0–4).	Table of §8.5 with the four examples.
Why can't NMI be masked? Where is it used?	It is a positive-edge, non-maskable input; uses: power failure, critical response times, non-recoverable hardware errors, watchdog, memory parity errors.

10. LAST-MINUTE CRAM SHEET (ARCHITECTURE)

Numbers to remember:

Item	Value
8086 released / word size	**1978 / 16-bit**
Transistors	**≈29,000**
Package / supply / clock duty cycle	**40-pin DIP / +5 V / 33 % (asymmetric)**
Address bus / memory	**20-bit / 1 MB (2^{20})**
Data bus	**16-bit**
Segment size / active segments	**64 KB / 4 segments = 256 KB**
Instruction queue	**6 bytes FIFO**
Physical address formula	**(Seg × 16_{10}) + Offset**
IVT location & size	**00000H–003FFH, 1 KB**
Interrupt vector size / count	**4 bytes / 256 vectors**
Vector address	**Type number × 4**
Interrupt groups	**5 dedicated (0–4), 27 reserved (5–31), 224 available (32–255)**
RESET must be high for	**at least 4 clock cycles**

Flags: CF bit 0 · PF 2 · AF 4 · ZF 6 · SF 7 · TF 8 · IF 9 · DF 10 · OF 11.

Modes: `MN/MX = 1` → **Minimum** (no coprocessor); `MN/MX = 0` → **Maximum** (multiprocessor, needs 8288).

Chips: 8284 = clock generator · **8282** = latch (address demux with ALE) · **8286** = transceiver (data bus) · **8288** = bus controller (max mode).

Registers: AX/BX/CX/DX (A=accumulator, B=base, C=count, D=data) · SP/BP · SI/DI · IP · CS/DS/SS/ES.

Two things you must never confuse: `MN/MX = 1` is **minimum**, and `INTR` is **maskable** while `NMI` is **not**.

Continue with Part 2 (Addressing Modes) and Part 3 (Instruction Set) — they carry the remaining exam questions of Module 2.